People's Democratic Republic of Algeria

Ministry of Higher Education and Scientific Research

National School of Cyber Security — Foundation Training Department

Technical Report — Mini-Project N°1

Automated Hardware & Software Audit System

Design and Implementation of an Automated Hardware & Software Audit System with Reporting and Remote Monitoring Capabilities

Module	Operating Systems
Academic Year	2025 / 2026
Supervised by	Sassi Bentrad
Developed by	Nessissen Mohamed — Boukhoulda Wissem

Table of Contents

1. Introduction
2. System Architecture and Design
3. Module Descriptions
4. Commands Used
5. Cron Configuration
6. Security Considerations
7. Challenges Encountered
8. Conclusion

1. Introduction

Throughout the OS course, one thing that kept coming up was how the system state evolves over time and why monitoring those changes is important. That discussion was what pushed us toward building something hands-on — a tool that does the data collection automatically so an admin does not have to repeat the same checks every day by hand.

What we came up with is SysAudit, a collection of Bash scripts that pull hardware and software information from a live Linux machine and write structured reports in four formats: plain text, HTML, JSON, and PDF. The tool works in three ways — through an interactive menu, as a nightly cron job, or pointed at a remote machine over SSH without needing to install anything on the target first.

This document walks through how we organized the code, what each script is responsible for, the Linux commands we used and why, how the cron scheduling works, the security features we added, and the issues we ran into during development.

2. System Architecture and Design

2.1 Design Approach

From the start we chose to split the code across separate files, one per concern, rather than dumping everything into a single script. This made it easier for both of us to work on different pieces in parallel and to test each module on its own. It also fits the standard Unix philosophy of keeping programs small and focused, each doing one well-defined job.

2.2 Directory Layout

```
B3_nessissen_OS/
├── main.sh        ← entry point; only file the user runs
├── utils.sh       ← color variables and print helpers
├── log.sh         ← logging and log rotation
├── hardware.sh    ← CPU, RAM, Disk, GPU, Network, Motherboard, USB
├── software.sh    ← OS info, packages, users, processes, services, ports
├── report.sh      ← report generation (TXT / HTML / JSON / PDF)
├── email.sh       ← email sending via msmtp or mailutils
├── security.sh    ← SHA-256 hashing, verification, report diff
├── remote.sh      ← remote audit over SSH
├── logs/          ← auto-created; contains audit.log and cron.log
└── reports/       ← auto-created; organized by format
    ├── txt/
    ├── html/
    ├── json/
    └── pdf/
```

2.3 Module Load Order

When main.sh runs, it first resolves its own absolute path using BASH_SOURCE[0] and stores it in $SCRIPT_DIR, which all sourced modules inherit. Then it sources the other files in this fixed order:

```
utils.sh → log.sh → security.sh → hardware.sh → software.sh → report.sh →
email.sh → remote.sh
```

utils.sh has to come first since every other module relies on its print helpers. log.sh comes second because all modules call log_msg(). The remaining files are sourced in logical dependency order after that.

2.4 Execution Modes

Interactive mode starts when you run bash main.sh with no arguments. An ASCII banner loads first, then a numbered menu appears. The loop stays open after each action so the user can run several operations without restarting the script.

Automated mode is activated by passing the --auto flag. The menu is bypassed completely. The tool generates a full report in all four formats, calculates the SHA-256 hash, logs the run, and exits cleanly. This is the mode the cron job always uses.

2.5 Report Types

Short report — a quick snapshot covering OS version, kernel, CPU model, RAM totals, root disk usage, installed package count, and current CPU usage. It finishes in a few seconds.

Full report — a complete audit of everything hardware.sh and software.sh can collect. It also triggers an alert email automatically if CPU usage is above 80% at the time of the run.

3. Module Descriptions

Module	Responsibility
`utils.sh`	Defines ANSI color codes and helper functions (print_info, print_error, etc.). Contains get_cpu_usage() which reads /proc/stat twice with a 0.5 s pause to compute real usage.
`log.sh`	Provides log_msg() which appends timestamped lines to audit.log. rotate_logs() compresses the file once it hits 100 KB and keeps at most 5 backup archives.
`hardware.sh`	Collects CPU via lscpu, GPU via lspci, RAM via free and /proc/meminfo, disk via lsblk and df, network via ip, motherboard via dmidecode, USB via lsusb.
`software.sh`	Collects OS info from uname and /etc/os-release, installed packages via dpkg/rpm/pacman, logged-in users via who and last, processes via ps, services via systemctl, open ports via ss.
`report.sh`	Builds TXT, HTML (dark-themed), JSON (jq-escaped), and PDF reports using a fallback chain: LibreOffice → wkhtmltopdf → pandoc → enscript+ps2pdf. Handles both short and full report variants.
`security.sh`	generate_hash() fingerprints a report with SHA-256. verify_hash() re-checks it later. compare_reports() runs diff between two audit files to detect changes.
`email.sh`	send_email() pipes a report to msmtp or mail. send_alert_email() fires an urgent notification when CPU usage exceeds 80%.
`remote.sh`	SSHs into the target host, creates /tmp/audit_scripts/, copies all modules with scp, runs --auto remotely, and pulls the reports back via a tar pipe over SSH.

4. Commands Used

4.1 Hardware Data

Command	Purpose
`lscpu`	CPU model, core count, threads, frequency
`lspci | grep -iE 'vga|3d|display'`	GPU / display adapter info
`free -h`	RAM and swap totals in human-readable form
`grep -E '^(MemTotal|...)' /proc/meminfo`	Fine-grained kernel memory values
`lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINT`	Block device tree
`df -h --output=source,size,used,avail,pcent,target`	Partition usage
`ip -brief addr show`	Network interfaces and IP addresses
`ip link show | awk '/link\/ether/...'`	MAC addresses
`ip route`	Routing table and default gateway
`dmidecode -t baseboard`	Motherboard manufacturer and model
`lsusb`	Connected USB devices

4.2 Software and OS Data

Command	Purpose
`uname -a / -r / -m`	Kernel version and architecture
`cat /etc/os-release`	Distribution name and version
`dpkg -l | grep '^ii'`	Installed packages (Debian/Ubuntu)
`rpm -qa / pacman -Q`	Installed packages (RPM / Arch)
`who`	Currently logged-in users
`last -n 10`	Last 10 login events from /var/log/wtmp
`ps aux --sort=-%cpu | awk 'NR==1||NR<=21'`	Top 20 processes by CPU usage
`systemctl list-units --type=service --state=running`	Active system services
`ss -tuln`	Listening TCP/UDP sockets

4.3 Security, Scheduling, and Remote Commands

Command	Purpose
`sha256sum <file>`	Generate a SHA-256 fingerprint of a file
`sha256sum -c <file>.hash`	Verify a file against its saved hash
`diff <old> <new>`	Compare two report files line by line
`jq -R -s '.'`	Escape arbitrary text for valid JSON embedding
`sed -r "s/\x1B\[...//g"`	Strip ANSI color codes before writing to files
`awk '/^cpu / {print $2+...}' /proc/stat`	Read CPU tick counters to compute usage
`crontab -l`	Display the current user's crontab
`(crontab -l; echo "...") | crontab -`	Safely append a cron entry without duplicates
`ssh <user>@<host> "mkdir -p /tmp/audit_scripts"`	Prepare the workspace directory on the remote machine
`scp -q "$SCRIPT_DIR"/*.sh <user>@<host>:/tmp/audit_scripts/`	Push all audit modules to the remote machine
`ssh ... 'tar -cf - -C /tmp/audit_scripts/reports .' | tar -xf -`	Download remote reports via a tar pipe over SSH

5. Cron Configuration

5.1 Schedule

We set the tool to run automatically once a day at 4:00 AM. The crontab line we added is:

```
0 4 * * * /tmp/sysaudit_cron_wrapper.sh
```

Cron Field	Value	Meaning
Minute	0	At the zeroth minute (start of the hour)
Hour	4	04:00 AM
Day of Month	*	Every day
Month	*	Every month
Day of Week	*	Any day of the week

5.2 The Wrapper Script

Our project folder name has a space in it, and that caused cron to fail even when we tried quoting the path — cron splits the command line before the shell ever gets to interpret the quotes. To work around it, we placed a small wrapper script at /tmp/sysaudit_cron_wrapper.sh (no spaces anywhere in that path). Cron calls the wrapper; the wrapper handles the cd into the project directory and then calls main.sh --auto correctly.

```
#!/bin/bash
cd "/home/medness/Desktop/B3_nessissen_OS"
bash "/home/medness/Desktop/B3_nessissen_OS/main.sh" --auto \
  >> "/home/medness/Desktop/B3_nessissen_OS/logs/cron.log" 2>&1
```

Both stdout and stderr are appended to logs/cron.log, so if something goes wrong at 4 AM we can look at that file in the morning and see exactly what happened.

5.3 Setup Procedure

The cron job is added through menu option 6 (Setup Cron Job). The tool first displays the current crontab, then previews the exact line it is going to add, and asks for confirmation. It also removes any existing SysAudit entries before appending the new one, so running it twice never creates a duplicate.

5.4 Verification

```
$ crontab -l
0 4 * * * /tmp/sysaudit_cron_wrapper.sh

$ cat /tmp/sysaudit_cron_wrapper.sh
#!/bin/bash
cd "/home/medness/Desktop/B3_nessissen_OS"
bash "/home/medness/Desktop/B3_nessissen_OS/main.sh" --auto \
  >> "/home/medness/Desktop/B3_nessissen_OS/logs/cron.log" 2>&1
```

6. Security Considerations

6.1 Report Integrity — SHA-256 Hashing

A report loses its value if it can be altered silently after the fact. Every time a full report is generated, generate_hash() immediately creates a SHA-256 fingerprint of the file and stores the digest alongside it as <report>.hash. When you want to confirm nothing changed, verify_hash() re-runs sha256sum -c against that saved digest. Any mismatch means the file was modified since it was written. SHA-256 produces a 256-bit digest, making a collision computationally infeasible under normal conditions.

6.2 Change Detection via Report Diff

compare_reports() runs a diff between any two reports from different audit runs. This makes it straightforward to notice whether new network ports appeared, packages were installed or removed, or user accounts were added between two points in time. The diff output is timestamped and kept for review.

6.3 Credential and Permission Handling

Email credentials go in ~/.msmtprc. That file must have permissions set to 600 — meaning only the owner can read or write it — so other accounts on the same machine cannot access it. For Gmail with two-factor authentication, we recommend using an App Password instead of the main account password.

6.4 SSH Remote Access

Remote auditing runs entirely over SSH, so both the control channel and the data transfer are encrypted. The tool supports both password authentication and SSH key pairs. Key-based auth is the better option for automated runs because you never have to embed a plain-text password anywhere. Audit modules are copied to /tmp/audit_scripts/ on the target, which is writable on virtually any Linux system without elevated privileges.

6.5 Privilege Approach

SysAudit runs without root access. The one exception is dmidecode, which needs elevated privileges to query motherboard information. The tool checks for sudo first and only calls dmidecode with it if available; otherwise it reads the same data from /sys/class/dmi/id/ directly, which does not require special permissions. Running the entire tool as root for routine audits is not recommended.

7. Challenges Encountered

7.1 Space in the Project Path

Honestly, this took the most time to sort out. The space in the folder name broke cron, caused Bash variable expansions to split paths at the wrong character, and made scp refuse to copy files. We fixed it two ways: wrapping every path variable in double quotes throughout the code, and using the /tmp wrapper so cron never has to see the path with the space.

7.2 Absolute Paths in Cron Context

A script that works perfectly from the terminal can break completely under cron, because cron runs with a stripped-down environment and its working directory is not the same as yours. Early on we used relative paths like ./logs/audit.log, and cron would silently write files to the wrong location or just fail without any error message. We solved this by making the script resolve its own directory at startup:

```
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
export SCRIPT_DIR
```

BASH_SOURCE[0] always contains the path of the script file itself, not the caller's current directory, so this gives the correct absolute path no matter how or where the script is launched.

7.3 PDF Generation Across Environments

There is no one PDF tool you can rely on across different Linux installations. LibreOffice produces clean output but takes a few seconds to start in headless mode. wkhtmltopdf is faster but not always installed. pandoc handles many formats but is optional. enscript paired with ps2pdf is more commonly available but only deals well with plain text. We handled this by checking for each tool in order and falling back to the next one: LibreOffice → wkhtmltopdf → pandoc → enscript+ps2pdf.

7.4 ANSI Color Codes in Output Files

The print helpers in utils.sh write colored output using ANSI escape sequences. That looks fine in a terminal, but those same sequences show up as garbage characters in text files and completely corrupt JSON strings. We added a strip_ansi() function that pipes all output through sed to remove the escape sequences before anything is written to disk.

7.5 JSON Escaping of Arbitrary Content

Real system output regularly contains backslashes, double quotes, newlines, and other characters that require escaping in JSON. A handful of sed substitutions are not enough to catch everything reliably. We switched to routing output through jq -R -s '.' right after the ANSI stripping step. jq handles all edge cases and always produces syntactically valid JSON, regardless of what the system returned.

8. Conclusion

This project pushed us to apply course concepts in a way that lab exercises usually do not. Writing SysAudit meant actually interacting with kernel interfaces, configuring the cron daemon, and managing remote machines over SSH. The codebase ended up at around 700 lines of Bash across nine files, covering hardware collection, four report formats, integrity verification, email alerts, log rotation, and remote auditing — all without pulling in any external language or framework.

The work made a few course topics click in a way that reading about them did not. Things like the difference between sourcing and executing a script, how cron's environment differs from an interactive shell, how the kernel exposes hardware details through /proc and /sys, how piping connects separate programs, and how a cryptographic hash functions as a real integrity check rather than just a theoretical concept.

The issues we hit along the way — the space in the folder name, cron's isolated environment, chasing down which PDF tool was available, ANSI codes leaking into files, and JSON special characters breaking output — are not the kind of thing that shows up in textbook problems. Working through each one required understanding how the shell and kernel actually behave, not just what the syntax looks like.